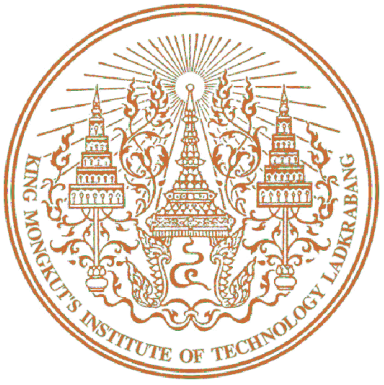
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
SCHOOL OF ENGINEERING



Data Structures and Algorithms

Project Report: LEBIS — Algorithms for an Equipment Rental System

INSTRUCTED BY: Dr. Anakkapon Saenthon

NAME	STUDENT ID
NITHIT SRISENGSUWANCHAI	68011411
INTUCH SRISAWAT	68011345
ISSAREE AUSAVAPIROM	68011346

DATE OF SUB.: 18/10/2026

Contents

Introduction 3

Part 1: Inserting and Deleting with a Hash Table (Kim) 5

Part 2: Searching and Counting with Binary Search (Neo) 10

Part 3: Sorting and Grouping with Merge Sort (Kul) 16

Conclusion: Three Algorithms Working Together 26

Appendix: Flowcharts 28

Introduction

This report was put together by Neo from the three parts written by each member. All the numbers in it come from a single run of `LEBIS_Final.ipynb`, the notebook that combines everyone's code.

The system these algorithms serve

LEBIS is a borrow-and-return system for small equipment rental shops, for example a camera rental shop run by one to three people. It is deployed at https://lebis-project.vercel.app.

A shop like this asks the same few questions every day, and each question suits a different data structure. In this report we look at which structure fits which question, and why.

Question from the shop	Algorithm group	Author	Part
"Is camera CAM-0012 available?", adding new stock, removing stock	Inserting, Deleting	Kim (hash table)	1
"The lens I rented before", "How many are free and how many are overdue?", "What is due back before date X?"	Searching, Counting	Neo (binary search)	2
"Which rental has been out the longest?", "What is in each category?"	Sorting, Grouping	Kul (merge sort)	3

Each of us implemented the main algorithm by hand. Python's `dict`, `bisect` and `sorted()` are not used to do the actual work. We only use them as baselines, to check that our results are correct and to compare speed.

All the code is in `backend-api/algorithms/` and imports only the Python standard library, so every file runs unchanged in Google Colab. Some of the functions are also used by the live web application: the counting functions and the range search from Part 2, and the sort by due date from Part 3 (Sections 2.11 and 3.8). The hash table from Part 1 is used in the notebook only.

The shared data sets

All three parts are measured on the same data from `make_dataset()` in `datasets.py`, so the results can be compared across parts. The data is generated from fixed seeds instead of new random values, and the reference date is fixed at `TODAY = 2026-10-01`, so every run produces exactly the same data.

Set	Size	Duplicate names	Out of stock	Overdue	Broken	Purpose
A	10	0	1	3	1	Check that every function works in the normal case
B	12	6	5	4	1	Duplicate names: search by code must return the right item, and search by name must return every match
C	10	0	5	4	1	Many items out of stock, to test status and counting at the edges
D	1,000	0	288	275	142	Show how the Big-O difference appears when the data is 100 times larger

Every set has at least 10 records, as the assignment requires. Each record has 8 fields: `code`, `name`, `category`, `total`, `available`, `borrower`, `due_date` and `status`.

How time was measured

All three parts use the same method: `time.perf_counter()`, run 3 times and averaged. The time spent preparing the data is not included. Times are in milliseconds.

The exact times change a little from run to run depending on the machine, but the order of which method is faster stays the same. This is why every number in the report comes from the same run. We did not mix in numbers from our individual notebooks.

Google Colab

The combined notebook, with its saved output: https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8

Please do not press Run all in the shared notebook. The saved output is the run used in this report, and running it again would change the times so they no longer match. To try the code, make a copy first (File > Save a copy in Drive).

Course requirements

Each algorithm has to include five things. The table shows the section where each one can be found.

Requirement	Part 1 (Kim)	Part 2 (Neo)	Part 3 (Kul)
Big-O for time and memory, with an explanation	1.5	2.6	3.5
Flowchart	1.8 and Appendix	2.9 and Appendix	3.10 and Appendix
Real results on sets A to D	1.7	2.8	3.7
Time on set D compared with what Python provides	1.7 (`dict`)	2.8 (scanning the whole list)	3.7 (`sorted()`)
Colab link	1.9	2.10	3.11

The Appendix at the end contains all seven flowcharts in figure order.

Part 1: Inserting and Deleting with a Hash Table (Kim)

Written by Kim, translated from the Thai original (`kim-hashtable.md`) for this report. All numbers come from `LEBIS_Final.ipynb`.

1.1 The process chosen

A small photography equipment rental shop with one to three staff deals with the same tasks every day:

- A customer asks whether camera CAM-0012 is available, so an item has to be found by its code.
- New stock arrives, so items have to be added, and two items must never share a code.
- Items are rented out and returned, so the number available has to change.
- An item breaks and cannot be repaired, so it has to be removed from the system.

If the items are kept in a plain list, each of these tasks means going through the list one item at a time until the right one is found, and this gets slower as the shop grows. This part therefore covers the Inserting and Deleting algorithm groups, including lookup by code. The hash table is written from scratch and is not a wrapper around Python's `dict`.

1.2 Data structure

Each item is stored as a record with 8 fields, the same format used by the whole team:

`code`, `name`, `category`, `total`, `available`, `borrower`, `due_date`, `status`

The system keeps the items in two places at the same time:

```
items = [ {CAM-0001 ...}, {LEN-0002 ...}, ... ]   records, in order
index = HashTable: "CAM-0001" -> 0, "LEN-0002" -> 1   code -> position
```

Stored as	Lookup by code	Sorting or going through in order
List only	O(n)	Yes
Hash table only	O(1) on average	No, the order is just the order of the buckets
List with a hash index	O(1) on average	Yes

We need both because other parts of the system depend on order. Kul's merge sort sorts rentals by due date, and Neo's binary search finds what is due before a given date. A hash table cannot answer range questions or give items back in order, so the list has to stay. The cost is the memory for one extra table, and the two have to be kept in sync at all times.

1.3 The hash table

Hash function

We use a polynomial rolling hash:

```
h = 0
for every character c in the code:   h = (h × 31 + ord(c)) mod 1,000,000,007
bucket = h mod number_of_buckets
```

The reasons for this choice are:

- The position of each character matters, so `CAM-01` and `CAM-10` end up in different buckets. If we simply added up the character codes, these two would get the same value, and real shop codes look exactly like this.
- 31 is a prime number. It is small enough to multiply quickly but large enough to spread the values out.
- Taking the result modulo a large prime stops the number from growing without limit, which would slow the calculation down.

Collisions and chaining

A collision happens when two different codes are mapped to the same bucket. Collisions cannot be avoided, since the number of possible codes is unlimited but the number of buckets is not. We handle them with chaining: one bucket can hold several pairs linked together as a chain, and a lookup goes to that bucket and checks the short chain.

Resizing

When the load factor (the number of items divided by the number of buckets) goes above 0.7, the table doubles its number of buckets and places every item again (rehashing). The load factor is the average length of a chain, so it tells us whether the table is still fast.

- If the threshold is too low, the table is mostly empty, wastes memory and resizes often.
- If it is too high, it saves space, but the chains get long and lookups slow down.

Each resize costs O(n), but it only happens when the amount of data has doubled. Averaged over all inserts, each insert still costs O(1) (amortized O(1)).

1.4 Adding, renting, returning and removing

Every function checks its input before changing anything, and if a check fails, the data is left untouched. If the list were changed first and the index update then failed, the two would stop matching for good.

Function	Rejected when
`add_item`	The code already exists, or the quantity is invalid
`borrow_item`	The code does not exist, the item is broken, no unit is available, or no renter name is given
`return_item`	The code does not exist, or the item is not currently rented
`remove_item`	The code does not exist, or some units are still with customers

To remove an item we use swap-and-pop. Deleting from the middle of a list is O(n), because every item after it has to move up one place, and the positions the index remembers for those items would all become wrong. Instead, we move the last item into the gap, update the index entry for that one item, and drop the

last slot. Every step is O(1). The downside is that the order of the list changes, but that is acceptable because any task that needs a particular order sorts the data itself.

1.5 Complexity (Big-O)

Operation	Average	Worst	The worst case happens when
Lookup by code	O(1)	O(n)	Every code lands in the same bucket
Insert	O(1) amortized	O(n)	The insert triggers a resize, or a chain is long
Delete	O(1)	O(n)	A chain is long
Resize	O(n) per resize	O(n)	(always)

O(1) is the average, not the worst case. If every code were hashed into the same bucket, the table would become one chain of length n, and a lookup would be no better than scanning a list. This can happen when the hash function spreads values badly, when the table never resizes, or when someone deliberately picks codes that all collide. Section 4 of the notebook shows a table that is not allowed to resize, for comparison. Its average chain is 15 items long and its longest is 17, while a table that resizes normally has a longest chain of only 3. The more data is added, the longer the chains in that table get, so in practice it becomes O(n). In other words, the O(1) average depends on having a hash function that spreads values well and on resizing. Without either one, it drops back to O(n).

1.6 Test data sets

We use the same `make_dataset()` (written by Neo) as the rest of the team. Every set has at least 10 records, as the assignment requires.

Set	Size	Characteristics	Used to show
A	10	Clean data	Every function works
B	12	Duplicate names, unique codes	Lookup by code returns the right item
C	10	Many items out of stock	Status and counting
D	1,000	Large set	The Big-O difference as the data grows

1.7 Results

We measured three methods on the same data: scanning a list, our hash table, and Python's `dict` as the baseline. Each cell was run 3 times and averaged with `time.perf_counter()`, and the time to prepare the data was not counted. Times are in milliseconds. "Speed-up" is the list scan time divided by our hash table time.

Table H1. Look up every code once

Set	List scan	HashTable	dict	Speed-up
A (10)	0.004	0.015	0.001	0.3×
B (12)	0.005	0.017	0.001	0.3×
C (10)	0.004	0.014	0.001	0.3×
D (1,000)	21.753	1.530	0.053	14.2×

Table H2. Add every record one at a time (every method checks for a duplicate code first)

Set	List scan	HashTable	dict	Speed-up
A (10)	0.004	0.042	0.002	0.1×
B (12)	0.005	0.066	0.002	0.1×
C (10)	0.004	0.041	0.002	0.1×
D (1,000)	20.625	5.902	0.112	3.5×

Table H3. Delete every code once

Set	List scan	HashTable	dict	Speed-up
A (10)	0.006	0.019	0.002	0.3×
B (12)	0.005	0.020	0.001	0.3×
C (10)	0.005	0.016	0.001	0.3×
D (1,000)	15.152	1.719	0.061	8.8×

Table H4. State of the hash table after inserting the whole set

Set	Items	Buckets	Collisions	Resizes	Longest chain	Load factor
A	10	16	2	1	2	0.625
B	12	32	5	2	3	0.375
C	10	16	2	1	2	0.625
D	1,000	2,048	374	8	3	0.488

Reading the results

1. On the small sets (A to C) the list scan is faster. Checking 10 items costs less than computing a hash for every lookup. Big-O describes how the time grows as n gets larger. It does not tell us which method wins when n is small, so if we had only measured the small sets we would have reached the wrong conclusion.
2. On set D the hash table is faster at all three tasks. Set D is 100 times larger than set A. The lookup time for the list scan grew about 5,438 times (n lookups, each checking half the list on average, so about $n^2/2$), while the hash table grew about 102 times, in line with n.
3. Inserting has the smallest speed-up (3.5×), because the hash table resized 8 times while the 1,000 records were added. This cost is spread over all the inserts (amortized), so the total still grows with n

rather than n^2.

4. Set D had 374 collisions, but the longest chain was only 3, because the table resized every time the load factor went over 0.7. This shows that collisions are not a problem as long as the chains are kept short.
5. Python's `dict` is faster than ours, about 28 to 29 times for lookup and delete and about 53 times for insert. `dict` resizes itself in C, while ours rehashes one item at a time in Python. Both are O(1), though, so they differ only by a constant factor and grow at the same rate.

The measured times change slightly every time the code is run, but the order of which method is faster stays the same.

1.8 Flowcharts

The flowcharts are in `docs/data-course/flowcharts/` as both draw.io and PDF files. The PDFs are also in the team's Drive folder: https://drive.google.com/drive/folders/1r2B5_h15vtCjMgca2rhnYVqcbX1wulWY

1. `flow_kim_1_overview`: an overview of the system, where every command goes through the hash index first
2. `flow_kim_2_insert`: adding an item, including what happens on a collision and when the table resizes
3. `flow_kim_3_delete_return`: removing an item with swap-and-pop, and returning an item

1.9 Related files

File	Description
`backend-api/algorithms/hash_table.py`	All of the code, using only the standard library
`docs/data-course/LEBIS_Final.ipynb`	The combined notebook of all three members, already run. The numbers in this report come from this file. Colab: https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8
`docs/data-course/LEBIS_Kim_HashTable.ipynb`	Kim's own notebook, with more detailed explanations. Its numbers come from a different run, so they should not be mixed with this report.
`scripts/data-course-hashtable.py`	Script that measures sets A to D locally
`scripts/build-notebooks.py`	Builds the notebooks from the actual files in the repository

Part 2: Searching and Counting with Binary Search (Neo)

Written by Neo. All numbers come from `LEBIS_Final.ipynb`.

2.1 The process chosen

Part 1 answered "where is the item with this code?" with a hash table, which is the fastest option when the exact code is known. In a real shop, though, the questions go further than that:

- A customer may not remember the code and only describe the item, for example "the lens I rented last time". Items then have to be found by part of their name, and names can repeat.
- When the owner opens the system in the morning, they want to know how many units are free, how many are with customers, and how many rentals are overdue.
- Before closing, they may ask what is due back before a certain date. This is a range question, not a lookup of one value.

A hash table cannot answer the last two at all, because it can only work out a position from an exact key. This part therefore covers the Searching and Counting algorithm groups, with binary search written by hand. Python's `bisect` is not used, except as something to compare against.

2.2 Data structure and the precondition for searching

The whole team uses the same 8-field record from `make_dataset()`:

`code`, `name`, `category`, `total`, `available`, `borrower`, `due_date`, `status`

Binary search needs one thing that a plain list does not guarantee: the data has to be sorted already. If the list is not sorted by `code`, the search cuts off the wrong half and returns `None` even though the item is in the list. Nothing reports this as an error, so someone would have to spot it by eye. For this reason we provide `sort_by_code()` to call first, and it returns a new list instead of changing the original.

In the live system, the list sorted by `due_date` comes directly from Kul's merge sort (`sort_by_due_date()`). Since our two parts are connected, we agreed that every function returns a new list and never changes a list that someone else is holding.

2.3 Three search paths

Which path is used depends on the kind of question, not on which method is fastest.

Path	Function	Used when	Time
Linear	`linear_search`	The list is not sorted and there is no hash table. Also used as the baseline	O(n)
Hash	`hash_lookup`	The exact code is known and Kim's table is available	O(1) on average
Binary	`binary_search`	Exact code on a sorted list (the one written by hand in this part)	O(log n)
Partial name	`search_by_name`	The customer only remembers the name	O(n)

Binary search starts by looking at the middle item. If it matches, the search stops. If the target is smaller than the middle item, it has to be in the left half, so the whole right half can be ignored, and the other way round. Each comparison cuts the remaining part in half, which means 1,000 items need only about 10 comparisons, since $2^{10} = 1{,}024$.

We wrote it as a loop rather than with recursion. A loop does not use extra stack space, so the memory is O(1) instead of O(log n). The middle is computed as `middle = low + (high - low) // 2` rather than `(low + high) // 2`. This is the usual way to avoid overflow in languages where `int` has a fixed size. Python does not overflow, but we kept the safer form anyway.

`search_by_name` has to return every match, not only the first one, because names really do repeat in a shop. If it returned the first match like `linear_search` does, the user could pick the wrong item without knowing it, rent out the wrong unit, and the other item would quietly drop out of view. Set B was built to test exactly this.

2.4 Range questions, where a hash table cannot help

`find_due_before(sorted_by_due, limit_date)` answers "what is due back before date X?".

It uses the same halving idea to find the leftmost position where `due_date >= limit_date`, which takes O(log n), and then takes every item before that position in one block. The total is O(log n + k), where k is the number of results. Those k results have to be copied out whichever method is used.

Note that `high` starts at `len()` and not `len() - 1`, because the answer can be "past the end of the list", meaning no item reaches the limit.

A hash table cannot do this. It only knows which bucket an exact key belongs to and has no idea of "before" or "after", so a range question would mean scanning the whole table in O(n). This is the reason the project still needs binary search even though it already has Kim's hash table.

2.5 Counting

The three counting functions have exactly the same structure: one pass over the data. They only differ in the condition they check and in what they add up.

Function	Counts	Easy to get wrong
`count_available`	Units free on the shelf	Broken units are not counted. They may still be on the shelf, but they cannot be rented
`count_by_status`	Records per status	It always returns all 3 keys, even when a count is 0
`count_overdue`	Rentals past their due date	Returned items have no `due_date`, so they must be skipped first, otherwise Python raises `TypeError`

All three are O(n), and there is no shortcut. Counting is different from searching because there is no single item to stop at, so every item has to be checked before the total is known. Binary search does not help, since halving needs one target that can be compared as larger or smaller. A hash table does not help either, because it answers "where is this item", not "how many items meet this condition".

`count_by_status` works out all three numbers in one pass instead of three separate loops. The result is the same, but the data is read once instead of three times.

Dates can be compared as plain strings because the format `YYYY-MM-DD` goes from the largest unit to the smallest. For example, `"2026-09-26" < "2026-10-01"` is true both as text and as dates. If the dates were stored as `DD/MM/YYYY`, this would no longer work.

2.6 Complexity (Big-O)

Function	Time	Memory	The worst case happens when
`linear_search`	O(n)	O(1)	The item is not in the list, so every item has to be checked
`binary_search`	O(log n)	O(1)	The item is at one end, so all $\log_2(n)$ rounds are used
`hash_lookup`	O(1) on average	O(n) for the table	Every code lands in the same bucket, which makes it O(n)
`search_by_name`	O(n)	O(k) for the results	Every record contains the search term
`sort_by_code`	O(n log n)	O(n)	(the same in every case)
`find_due_before`	O(log n + k)	O(k)	Every record matches (k = n)
`count_available`, `count_by_status`, `count_overdue`	O(n)	O(1)	Every case is the same, there is no shortcut

Binary search needs only O(1) memory even though it halves the list again and again, because it is a loop and not recursion. It only uses three variables, `low`, `high` and `middle`, no matter how much data there is. A recursive version would use one stack frame per level, so its memory would be O(log n).

2.7 Test data sets

The team shares the same `make_dataset()`, which was written as part of this work (`datasets.py`). The reference date is fixed at `TODAY = 2026-10-01`.

Set	Size	Duplicate names	Out of stock	Overdue	Broken	Used to show
A	10	0	1	3	1	Every function works in the basic case
B	12	6	5	4	1	`search_by_name` returns every match
C	10	0	5	4	1	Counting when many items are out of stock
D	1,000	0	288	275	142	The Big-O difference as the data grows

The data is generated from fixed seeds instead of new random values, so the numbers in this report come out the same on every run. A broken item is placed at every 7th position so that even the small sets have at least one. Otherwise the "skip broken items" condition in `count_available` would never be tested.

2.8 Results

Each cell was run 3 times and averaged with `time.perf_counter()`, the same method used in Parts 1 and 3. The time to prepare the data is not included. Times are in milliseconds, and "Speed-up" is the linear

time divided by the binary time.

Table S1. Look up every code once

Set	Size	Linear	Binary	Hash	Speed-up
A	10	0.004	0.006	0.016	0.6×
B	12	0.004	0.007	0.018	0.6×
C	10	0.003	0.006	0.015	0.6×
D	1,000	21.812	1.759	1.580	12.4×

About the Python baseline in this table: item 4 of `plan.md` gives each member's work a different baseline. It is `dict` for the hash table, `sorted()` for sorting, and scanning the whole list for searching. The Linear column in Table S1 is that baseline, a plain Python loop over the items with no data structure to help it. The Hash column is Kim's `HashTable` from Part 1, included to show where the third method stands on the same data.

Neo's own notebook (`LEBIS_Neo_SearchCount.ipynb`, section 6) also compares against Python's `bisect`. The times are in the same range, and `bisect` is slightly faster because it is written in C, not because its algorithm is better. Those numbers are not in this table because they come from a different run, and numbers from different runs cannot be compared fairly.

Table S2. Counting results

Set	Free on shelf (units)	ok	out	broken	Overdue
A	24	9	0	1	3
B	14	7	4	1	4
C	10	5	4	1	4
D	1,309	616	242	142	275

Table S3. Searching by a duplicated name (set B)

Searching for `"เลนส์ รุ่นที่ 1"` ("Lens model 1") returns 4 records: `LEN-0001` (0/2, out), `LEN-0002` (3/5, ok), `LEN-0011` (4/5, ok) and `LEN-0012` (0/5, out).

Two of these names match exactly. The other two contain the search term inside a longer name, since "model 11" also contains "model 1". This is the behaviour we want for the search box on the website, because users can type only part of a name and still find the item.

Table S4. A range question (set D)

For "which items are due back before 2026-10-01?", binary search finds 275 records and a one-by-one O(n) count also finds 275 records, so the two results are identical.

Reading the results

1. On the small sets (A to C), linear search is faster than binary search (0.004 compared with 0.006 ms). There is too little data for the halving to make up for the cost of working out the positions. O(log n) does not mean "always faster", it means "grows more slowly", so judging binary search from the small sets alone would be a mistake.

2. On set D, binary search is 12.4 times faster than linear search (21.812 compared with 1.759 ms), and the hash table is 13.8 times faster than linear search (1.580 ms).
3. The growth rates are very close to the theory. The data grows 100 times from A to D, and this test looks up every code, which is n lookups. We would therefore expect:
    - binary: n × O(log n), so it should grow about 100 × 3 = 300 times. It actually grew 1.759 ÷ 0.006 = 293 times.
    - hash: n × O(1), so it should grow about 100 times. It actually grew 1.580 ÷ 0.016 = 99 times.

    Both are close enough to the Big-O prediction that it is unlikely to be a coincidence.
4. Linear search grew less than the theory says. n × O(n) should grow 10,000 times, but we measured 21.812 ÷ 0.004 ≈ 5,453 times. This does not mean the theory is wrong. At n = 10 most of the time goes on fixed costs, such as calling the function and setting up the loop, rather than on the comparisons themselves. The small-set time is therefore too large to divide by, and the ratio comes out smaller. It is the same effect as in point 1, seen from a different side.
5. The hash table and binary search are very close on set D (1.580 compared with 1.759 ms) even though their Big-O classes are different, because $\log_2(1{,}000) \approx 10$ is still a small number. The gap would only become clear at a much larger n. In any case, speed was not the reason for choosing binary search. The reason is Table S4, which a hash table cannot answer.
6. There is no speed comparison for counting, because there is nothing to compare it with: every method has to check every item. Table S2 therefore shows that the numbers are correct rather than how long they took.

The measured times change slightly every time the code is run, but the order of which method is faster stays the same.

2.9 Flowcharts

The flowcharts are in `docs/data-course/flowcharts/` as both draw.io and PDF files. The PDFs are also in the team's Drive folder:
https://drive.google.com/drive/folders/1r2B5_h15vtCjMgca2rhnYVqcbX1wulWY

1. `flow_neo_4_search`: searching, showing all 3 paths, with the binary search loop drawn step by step
2. `flow_neo_5_count`: the 3 counting functions side by side, with the reason all of them are O(n)

2.10 Related files

File	Description
`backend-api/algorithms/search.py`	All of the searching and counting code, using only the standard library
`backend-api/algorithms/datasets.py`	Data sets A to D, shared by the whole team
`docs/data-course/LEBIS_Final.ipynb`	The combined notebook of all three members, already run. The numbers in this report come from this file. Colab: https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8
`docs/data-course/LEBIS_Neo_SearchCount.ipynb`	Neo's own notebook, with more detail. Its numbers come from a different run, so they should not be mixed with this report.
`scripts/data-course-benchmark.py`	Script that measures sets A to D locally using all three modules together

2.11 This code is used in the live system

The Workspace page of the website shows three numbers, and two of them are calculated directly by functions from this part, through `services/inventory.py`:

- `available_units` comes from `count_available()`
- `overdue_borrows` comes from `count_overdue()`

We chose not to use SQL `COUNT` or `SUM` here, even though they would be faster. Counting a second time in SQL would mean defining "free" and "overdue" in two different places, and sooner or later the two definitions would drift apart without anyone noticing.

Before switching over, we checked that every number stayed the same, both on normal data and on edge cases: a broken item that still had available units, a rental due exactly today, and a rental that became overdue yesterday.

`find_due_before()` also runs every morning at 08:00, when the system sends the shop owner a LINE summary of items due back. The system first sorts the rentals with Kul's merge sort, then uses binary search to take out the overdue rentals together with those due today and tomorrow in one block. This is the same sequence described in the Conclusion.

The one function that is not used on the website is `binary_search()` for lookup by code. The website finds an item by its code through a database index instead, and `binary_search()` is only measured in the notebook, for comparison with Kim's hash table.

Part 3: Sorting and Grouping with Merge Sort (Kul)

Written by Kul, translated from the Thai original (`kul-sort-group.md`) for this report. The timing numbers come from sections 4 and 5 of `LEBIS_Final.ipynb`.

3.1 The process chosen

A rental shop with one to three staff also asks questions that Parts 1 and 2 do not answer. These are questions about the stock as a whole:

- "Which rental has been out the longest?" The rentals need to be sorted by due date.
- "Which item is running low and should be restocked first?" The items need to be sorted by how many are available.
- "Which cameras does the shop have, and which lenses?" The items need to be separated by category.
- "What is broken at the moment?" The items need to be separated by status.

This part therefore covers the Sorting and Grouping algorithm groups. Which one we use depends on the kind of field:

Field	Kind of value	What we do	Why
`due_date`	A date, so one comes before another	Sort	"Out the longest" means the smallest value, so order matters
`available`	A whole number	Sort	"Running low" means the smallest value, so order matters
`category`	A label, such as camera or lens	Group	A camera does not come before or after a lens. There is no order, the items only need to be kept together
`status`	A label: `ok`, `out` or `broken`	Group	Same as above. We want to know what is in each status, not which status comes first

The algorithm written by hand in this part is merge sort. Python's `sorted()` and `.sort()` are not used to do the work. Bubble sort is also included as a comparison, to show how far apart $O(n^2)$ and $O(n \log n)$ are as the data grows, and `sorted()` is only used as a baseline.

3.2 Data structure

The input is the same 8-field record used by the whole team:

`code`, `name`, `category`, `total`, `available`, `borrower`, `due_date`, `status`

Function	Takes	Returns
`bubble_sort`, `merge_sort`	A list of records and a `key` function, like `sorted()`	A new sorted list
`sort_by_due_date`, `sort_by_available`	A list of records	A new sorted list
`group_by_type`, `group_by_status`	A list of records	A `dict` whose keys are category or status names and whose values are lists of the records in each group

The functions always return a new list and never change the original, because someone else might be holding the same list. This is the same approach as `sort_by_code` in Neo's `search.py`. It also means the three methods can be timed on the same list, since the first method does not leave the data already sorted and give the next method an unfair advantage.

The file only imports the Python standard library and does not import any other team member's file. All data is passed in as parameters, so the same file runs both in the web application and in Google Colab.

3.3 Sorting

Merge sort: split in half, then merge

```
merge_sort(items, key):
    if len(items) ≤ 1:  return list(items)       ← one item is always sorted (base
case)
    middle = len(items) // 2
    left  = merge_sort(items[:middle], key)      ← sort the left half
    right = merge_sort(items[middle:], key)      ← sort the right half
    return _merge(left, right, key)
```

`_merge` combines two sorted lists into one list that is still sorted. It compares the first remaining item on each side and takes the smaller one. When one side runs out, whatever is left on the other side is already in order, so it can be added to the end as it is.

```
i = j = 0, merged = []
while i < len(left) and j < len(right):
    if key(left[i]) <= key(right[j]):   take left[i],  then i += 1
    otherwise:                          take right[j], then j += 1
add left[i:] and right[j:] to the end of merged
```

We wrote it with recursion because splitting in half again and again fits that style naturally. The recursion goes about $\log_2(n)$ levels deep, which is about 10 levels for 1,000 records, so there is no risk of a stack overflow.

Stable sort: why <= and not <

A sort is stable if items with equal values stay in the same order they were in before sorting.

In `_merge`, the left side always holds the items that came first in the original data. When two values are equal, `<=` takes the item from the left side first, so the original order is kept. If we wrote `<` instead, the right item would be taken first and equal items would swap places for no reason. This one character is what makes our merge sort stable.

In the real system this means rentals due on the same day stay in the order the database returned them (by id), instead of changing places every time the page is opened. `verify()` in the file tests this with equal values and compares the result with `sorted()`, which is stable.

`bubble_sort` is also stable, because it only swaps when the first item is greater than the next one (`>`). Equal values are never swapped.

Bubble sort: the comparison

```
result = list(items)                        ← copy the list first, so the original is
not changed
for each round round_index = 0 … n-1:
    swapped = False
    for i = 0 … n - round_index - 2:
        if key(result[i]) > key(result[i+1]):  swap them, then swapped = True
    if nothing was swapped in this round:  stop    ← the list is already sorted
```

In each round the largest remaining value moves to the end, so the next round can stop one item earlier. Because it stops as soon as a round makes no swaps, data that is already sorted finishes after one round.

Functions the system calls

Function	What it does
`sort_by_due_date(items)`	Removes records whose `due_date` is `None`, then calls `merge_sort` with `key = due_date`, smallest first. The rental that has been out the longest (the oldest date) ends up at the top
`sort_by_available(items)`	The same `merge_sort` with `key = available`. The item with the fewest units left ends up at the top

`sort_by_due_date` has to remove `None` values first. An item nobody has rented has no due date to sort by, and Python cannot compare `None` with a string (it raises `TypeError` straight away). The result therefore only contains rentals that have a due date, which is 685 of the 1,000 records in set D.

`due_date` is stored as text in the form `YYYY-MM-DD`. Sorted as text, this gives the same order as the real dates (year, then month, then day), so the dates can be compared with `<=` directly without converting them first.

3.4 Grouping

Both grouping functions go through the data once and put each record into a bucket according to its key.

```
group_by_type(items):
    groups = {}
    for each item:
        groups.setdefault(category, []).append(item)
    return groups

group_by_status(items):
    groups = {"ok": [], "out": [], "broken": []}
    for each item:
        if status is in groups:  groups[status].append(item)
    return groups
```

	group_by_type	**group_by_status**
Are the keys known in advance?	No. A new key is created the first time a category appears (`setdefault`)	Yes. There are only 3 statuses, all created at the start
Keys returned	Only the categories that actually appear in the data	Always all 3 keys, even if a group is empty
Unknown values	(not applicable)	Skipped, so nothing breaks

`group_by_status` always returns all 3 keys for the same reason as Neo's `count_by_status`: code that uses the result does not have to guard every access with `.get()`.

The difference from Neo's `count_by_status` is that `count_by_status` only keeps the number of records, while `group_by_status` keeps the records themselves. It is meant for when we need to know what is in each group, not just how many.

We go through the data once instead of searching one category at a time. If we first searched for all the cameras and then searched again for the lenses, we would read the whole data set once for each category. The team's data has 5 categories, so that would mean reading it 5 times, while putting items into buckets reads it only once however many categories there are.

3.5 Complexity (Big-O)

Here n is the number of records, m is the number of rentals that have a `due_date`, and k is the number of categories.

Function	Time (best)	Time (average)	Time (worst)	Extra memory
`bubble_sort`	O(n)	$O(n^2)$	$O(n^2)$	O(n), because it copies the list before sorting
`merge_sort`	O(n log n)	O(n log n)	O(n log n)	O(n), plus O(log n) for the stack
`sort_by_due_date`	O(n + m log m)	O(n + m log m)	O(n + m log m)	O(m)
`sort_by_available`	O(n log n)	O(n log n)	O(n log n)	O(n)
`group_by_type`, `group_by_status`	O(n)	O(n)	O(n)	O(n)
Searching one category at a time (for comparison)	O(n × k)	O(n × k)	O(n × k)	O(n)

Merge sort is O(n log n) in every case

- Number of levels: the list is halved each time until one item is left, n → n/2 → n/4 → … → 1, which takes $\log_2(n)$ steps.
- Work per level: across one level, all the `_merge` calls together handle each of the n items exactly once, which is O(n).
- In total this gives O(n) per level × $\log_2(n)$ levels = O(n log n).
- There is no separate best or worst case because `merge_sort` always splits at `len(items) // 2` and never checks whether the data is already sorted. Sorted data is still split and merged at every level, so the number of levels is always the same.
- Memory is O(n). `_merge` builds a new `merged` list, and `items[:middle]` and `items[middle:]` copy the data into new lists. At any moment, the lists still in use belong to the levels currently in progress, with sizes n/2 + n/4 + …, which add up to only a few times n. The recursion is $\log_2(n)$ levels deep, so the stack is O(log n).

Bubble sort is $O(n^2)$ on average and in the worst case, and O(n) in the best case

- Worst case: the first round compares n−1 pairs, the second compares n−2 pairs, and so on. In total that is (n−1) + (n−2) + … + 1 = n(n−1)/2, which is $O(n^2)$.
- Best case: if the data is already sorted, the first round compares n−1 pairs without making any swap, so the loop stops straight away. This is O(n). Without this early stop, bubble sort would always run every round and be $O(n^2)$ even on sorted data.
- Memory is O(n), not O(1). The textbook version of bubble sort swaps in place and only needs O(1) extra memory. Our version starts with `result = list(items)`, which copies the whole list so the original is not changed, so it uses O(n) extra memory. The swaps after that are done in place on the copy.

sort_by_due_date is O(n + m log m)

Removing the `None` values with a list comprehension reads all n records, which is O(n), and produces a new list of m rentals, which is O(m) memory. Calling `merge_sort` on those m rentals is O(m log m), so the total is O(n + m log m). When m is close to n this is the same as O(n log n). In set D, m = 685 and n = 1,000.

Grouping is O(n)

- Time: there is one pass of `for item in items`, and each record takes a constant amount of work, which is finding its bucket in a `dict` (O(1) on average) and calling `append` (O(1) amortized). So n records take O(n), however many categories there are.
- Memory is O(n). Every record goes into a bucket and none are thrown away, apart from unknown statuses in `group_by_status`. The buckets hold references to the original records rather than copies. This is different from Neo's counting functions, which only need O(1).
- Searching one category at a time reads all n records k times, which is O(n × k). With the team's k = 5, that is five times as much reading as the bucket method.

3.6 Test data sets

We use the same `make_dataset()` (written by Neo) as the rest of the team. Every set has at least 10 records, as the assignment requires.

Set	Size	Characteristics	Used to show
A	10	Clean data	Every function works
B	12	Duplicate names, unique codes	Duplicate names make no difference, because names are not used as a key for sorting or grouping
C	10	5 of the 10 items are out of stock	Many records have `available = 0`, so there are many equal values
D	1,000	Large set, with 685 records that have a `due_date`	The Big-O difference as the data grows

3.7 Results

We measured three methods on the same data: bubble sort, our merge sort, and Python's `sorted()` as the baseline. Each method sorts the whole set once by the `available` field (`report_on_datasets` uses `key = item["available"]`), not by `due_date`. Each cell was run 3 times and averaged with `time.perf_counter()`, and the time to prepare the data was not counted. Times are in milliseconds, and "Speed-up" is the bubble sort time divided by the merge sort time.

Table M1. Sort the whole set once by `available`

Set	Bubble sort	Merge sort	`sorted()`	Speed-up
A (10)	0.011	0.013	0.001	0.9×
B (12)	0.014	0.015	0.001	0.9×
C (10)	0.008	0.013	0.001	0.6×
D (1,000)	90.516	2.619	0.124	34.6×

`sort_by_due_date`, which the system actually uses, is only called on rentals that have a `due_date` (685 in set D). The notebook does not time this function on its own, so the numbers in Table M1 are for `merge_sort` on the whole set by `available`.

Table G1. Grouping by category (`group_by_type`): number of records in each group

The category values in `datasets.py` are stored in Thai. The column headings give their meaning in English.

Set	Camera	Lens	Tripod	Light	Microphone	Total
A (10)	2	2	2	2	2	10
B (12)	2	4	2	2	2	12
C (10)	2	2	2	2	2	10
D (1,000)	200	200	200	200	200	1,000

Table G2. Grouping by status (`group_by_status`): number of records in each group

Set	ok	out	broken	Total
A (10)	9	0	1	10
B (12)	7	4	1	12
C (10)	5	4	1	10
D (1,000)	616	242	142	1,000

The figures for sets A to C come from running `group_by_type` and `group_by_status` on `make_dataset()` in the repository (the seeds are fixed, so the result is the same on every run). The figures for set D match section 4 of the notebook.

Reading the results

1. On the small sets (A to C), bubble sort is about as fast as merge sort or faster (0.9×, 0.9× and 0.6×). Merge sort has some fixed costs at every level: slicing creates new lists, the function calls itself again, and it builds a new `merged` list. With only 10 records, the real work is smaller than these costs, while bubble sort copies the list once and then swaps in place. Big-O describes how the time grows as n gets larger, not which method wins when n is small, so measuring only the small sets would lead to the wrong conclusion.
2. On set D, merge sort is 34.6 times faster than bubble sort (90.516 compared with 2.619 ms). The data grows 100 times from set A to set D. Bubble sort's time grows from 0.011 to 90.516 ms, which is

several thousand times. By the n^2 theory, 100 times more data should take about 10,000 times as long. Merge sort grows from 0.013 to 2.619 ms, which is in the hundreds, below the 300 times that the n log n theory gives. (The times for the small sets are only a few microseconds, so these ratios are rough and only show the general trend.)

3. `sorted()` is the fastest on every set. On set D it takes 0.124 ms, compared with 2.619 ms for our merge sort. `sorted()` uses Timsort, which is written in C, while our merge sort runs every step as a Python loop. Ours also calls `key()` again on every comparison, whereas `sorted()` works out each key only once per record. Both are still O(n log n), so the difference is a constant factor and not a difference in Big-O. They grow at the same rate as the data gets larger.
4. Grouping does not lose or add any records. Set D splits into 5 categories of 200 each, which is 1,000 in total, and by status into 616 + 242 + 142 = 1,000. The status counts are the same as Neo's `count_by_status` from the same run (Table 2 in the notebook) in all three columns. All of this is done in a single pass over the data. Searching one category at a time would need to read the data 5 times to get the same result.

The measured times change slightly every time the code is run, but the order of which method is faster stays the same.

3.8 Working together with Neo's and Kim's parts

Section 5 of the notebook answers a question the shop asks every morning: "Which rentals are overdue, and what is each item?" Answering it needs all three parts, one after another. Each step passes a plain list to the next, and none of us imports anyone else's file.

Step	Who	What it does	Time
1	Kul, `sort_by_due_date`	Sorts the rentals by due date	O(n + m log m)
2	Neo, `find_due_before`	Takes out every rental due before today using binary search	O(log m + k)
3	Kim, hash table	Looks up the details of each item by its code	O(1) per item

On set D, 685 rentals are sorted, and 275 of them are overdue before 2026-10-01. These are exactly the same rentals that a one-by-one scan finds. The notebook cell checks this with an `assert`, so it would fail straight away if the two did not match.

Step 1 has to be correct because Neo's binary search only works on a sorted list. It looks at the middle item and throws away half of the list, assuming that every item in that half is on the same side. If our part sorted the list wrongly, the search would cut off the wrong half and return the wrong rentals, and no error would be raised. This is why `verify()` compares `merge_sort` with `sorted()` and checks that `sort_by_due_date` really sorts from smallest to largest.

In the live web application, `services/inventory.py` calls `sort_by_due_date` in `open_borrows()` to sort the rentals that have not been returned yet (the SQL query does not sort them). The sorted list is then used in two places: Neo's `find_due_before` picks the rentals for the 08:00 LINE summary, and Neo's `count_overdue` counts the overdue rentals for the Workspace page. Step 3, the hash table, is only in the notebook and is not used by the web application.

3.9 Limitations

- Merge sort does not take advantage of data that is already sorted. It still splits and merges everything in O(n log n), while the Timsort used by `sorted()` can detect parts that are already in order and skip that work.
- It uses O(n) extra memory, because every level creates new lists by slicing and merging. An in-place sort would not need this.
- `key()` is called on every comparison instead of being worked out once and stored, as `sorted()` does. If the key were expensive to compute, the sort would slow down accordingly.
- `due_date` must be in the format `YYYY-MM-DD`. In another format, such as `DD/MM/YYYY`, comparing the text would give the wrong order without any error, and Neo's binary search would then also give wrong results.
- `sort_by_due_date` only returns rentals that have a `due_date`, so the result has m records rather than n. Whoever calls it needs to know that items nobody has rented are not included.
- `group_by_status` skips unknown statuses without any warning. If the data contains a wrong status, the three groups will add up to fewer records than the total and nothing will report it.
- The groups hold references, not copies, so changing a record inside a group also changes the original record.
- `bubble_sort` (only used for comparison), `sort_by_available`, `group_by_type` and `group_by_status` are not used in the live web application yet. They are only used in the notebook. The web application only uses `sort_by_due_date`.
- We do not have timing numbers for grouping or for `sort_by_due_date`, because the notebook only times sorting the whole set by `available` (Table M1).

3.10 Flowcharts

The flowcharts are in `docs/data-course/flowcharts/` as both draw.io and PDF files. The PDFs are also in the team's Drive folder:
https://drive.google.com/drive/folders/1r2B5_h15vtCjMgca2rhnYVqcbX1wulWY

6. `flow_kul_6_merge_sort.pdf`: merge sort. It checks the base case, splits the list in half and calls itself on both halves, then runs the `_merge` loop that compares the two front items with `<=` (the part that makes it stable) and adds whatever is left. A box at the side shows `sort_by_due_date`, the entry point the system actually calls, which removes `None` values and passes the result to Neo's `find_due_before`.
7. `flow_kul_7_group.pdf`: the two grouping functions side by side. `group_by_type` creates a new key when it meets a new category, and `group_by_status` creates all 3 keys at the start and skips unknown statuses. Both go through the data once.

The English versions used in this report are `flow_kul_6_merge_sort_eng.pdf` and `flow_kul_7_group_eng.pdf`.

3.11 Related files

File	Description
`backend-api/algorithms/sort_group.py`	All of the code, using only the standard library. It can be run on its own with `python3 backend-api/algorithms/sort_group.py`, which runs `verify()` first
`backend-api/services/inventory.py`	Where the live web application calls `sort_by_due_date` (in `open_borrows()`)
`docs/data-course/LEBIS_Final.ipynb`	The combined notebook of all three members, already run. The numbers in this report come from sections 4 and 5 of this file. Colab: https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8
`docs/data-course/flowcharts/flow_kul_6_merge_sort.pdf`, `flow_kul_7_group.pdf`	Both flowcharts, each with a matching `.drawio` file
`scripts/build-notebooks.py`	Builds the notebooks from the actual files in the repository

Conclusion: Three Algorithms Working Together

This conclusion was written by Neo as the editor and summarises the three parts above. All the numbers come from `LEBIS_Final.ipynb`.

One question that needs all three parts

Every morning the shop needs to know which rentals are overdue and what each of those items is. None of the three algorithms can answer this alone. It takes three steps, and each step passes its result to the next.

Step	Who	What it does	Time
1	Kul (merge sort)	Sorts all rentals by due date	O(n log n)
2	Neo (binary search)	Takes out every rental due before today in one block	O(log n + k)
3	Kim (hash table)	Looks up each item's details by its code	O(1) per item

Set D has 685 rentals. These three steps find 275 rentals that were overdue before 2026-10-01, and they are exactly the same rentals that a full one-by-one scan finds.

Each step passes a plain list to the next one, and none of us imports anyone else's code. If one of us changed the format of the data being passed on, this notebook cell would fail straight away, so it works both as an example of how the parts are used and as a test that they still fit together.

The live web application uses the first two steps in the same order. Every morning at 08:00 it sorts the rentals with merge sort, uses binary search to find what is due, and sends the shop owner a summary on LINE. It does not need step 3, because it reads each item's details from the database.

Main results

The table compares each hand-written algorithm with the simplest method that does the same job, on set D (1,000 records).

Part	Task	Simple method	Hand-written	Speed-up	Python's built-in (baseline)
1 (Kim)	Look up every code once	List scan 21.753 ms	Hash table 1.530 ms	14.2×	`dict` 0.053 ms
2 (Neo)	Look up every code once	One-by-one scan 21.812 ms	Binary search 1.759 ms	12.4×	Scanning the whole list (the simple method column)
3 (Kul)	Sort the whole set once	Bubble sort 90.516 ms	Merge sort 2.619 ms	34.6×	`sorted()` 0.124 ms

Complexity of every algorithm in the report

Part	Algorithm	Time (average)	Time (worst)	Memory
1	Lookup, insert and delete with the hash table	O(1)	O(n)	O(n)
2	Binary search	O(log n)	O(log n)	O(1)
2	Range query `find_due_before`	O(log n + k)	O(log n + k)	O(k)
2	Counting (`count_*`)	O(n)	O(n)	O(1)
3	Merge sort	O(n log n)	O(n log n)	O(n)
3	Grouping (`group_by_*`)	O(n)	O(n)	O(n)

The reasoning behind each value is explained in the Big-O section of the matching part.

What we found across all three parts

On small data, the simple method wins. All three parts saw this on sets A to C (10 to 12 records): the list scan beat the hash table, the one-by-one scan beat binary search, and bubble sort was as fast as merge sort or faster. With so few items, the fixed cost of the clever part (hashing, finding the middle, splitting the list) is larger than the actual work. Big-O tells us which method grows more slowly, not which one is faster for every n. If we had only measured the small sets, all three parts would have come to the wrong conclusion.

On large data, the Big-O difference shows up clearly in the measurements. From set A to set D the data grows 100 times. The test in Part 2 looks up every code in the set, so it makes n lookups, not one. Binary search grew 293 times, close to the 300 times predicted for n lookups at O(log n) each (100 × 3), and the hash table grew 99 times, close to the 100 times predicted for n lookups at O(1) each. A single lookup grows much less: about 3 times for binary search, and not at all for the hash table.

Python's built-in tools were faster than our versions because they are written in C, not because they use a better algorithm. On set D, `dict` and `sorted()` were many times faster than our hash table and our merge sort. They are in the same Big-O class, though, so they grow at the same rate as the data and differ only by a constant factor. (Neo's own notebook also compared `bisect` with our binary search, and `bisect` was only slightly faster.)

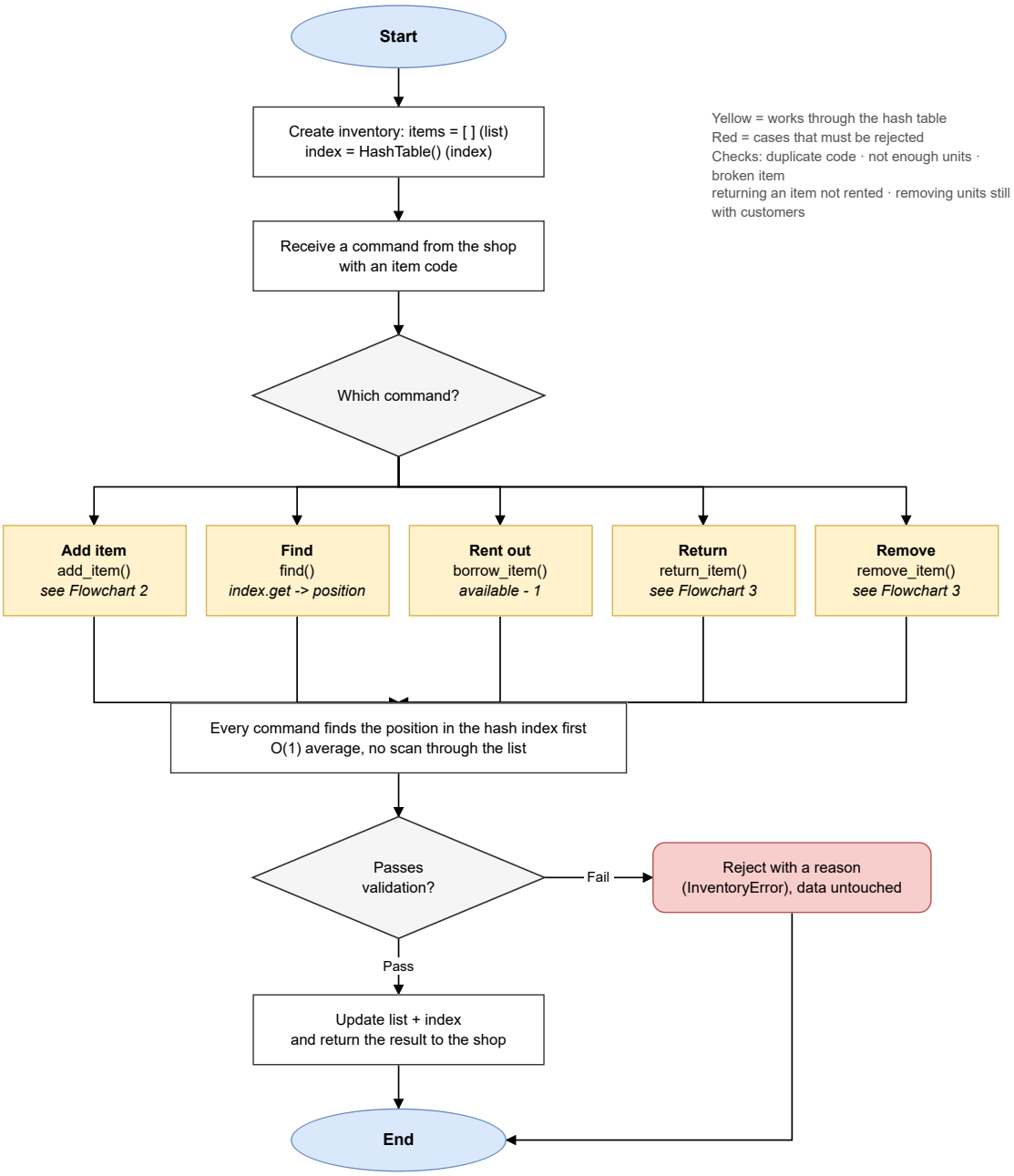
No single data structure can answer every question. A hash table is the fastest when the exact code is known, but it cannot answer range questions and has no order. A sorted list can answer range questions, but it has to be sorted first, which is what merge sort is for. This is why the inventory in our notebook keeps both and picks one depending on the question. In the web application, the database index plays the role of the hash table when an item is looked up by its code.

Appendix: Flowcharts

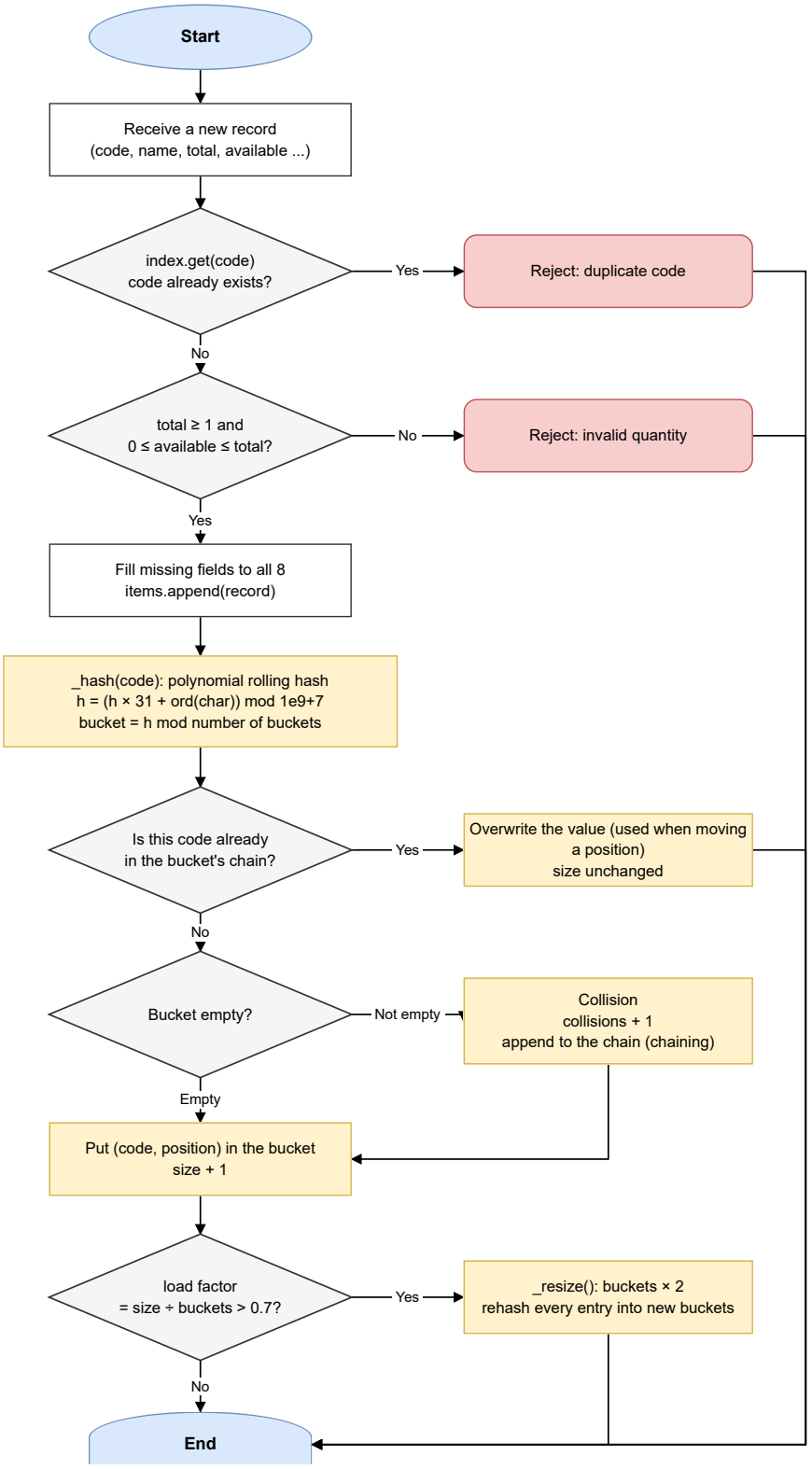
All flowcharts in the report, in figure order, one per page. The draw.io sources and PDFs are in `docs/data-course/flowcharts/`.

Figure	Subject	Author
1	System overview: inventory (list + hash index)	Kim
2	Adding an item (add_item + HashTable.insert)	Kim
3	Removing an item (remove_item) and returning (return_item)	Kim
4	Search: three paths (hash · binary · linear)	Neo
5	Counting: count_available · count_by_status · count_overdue	Neo
6	Sorting: merge_sort (split in half → merge)	Kul
7	Grouping: group_by_type and group_by_status	Kul

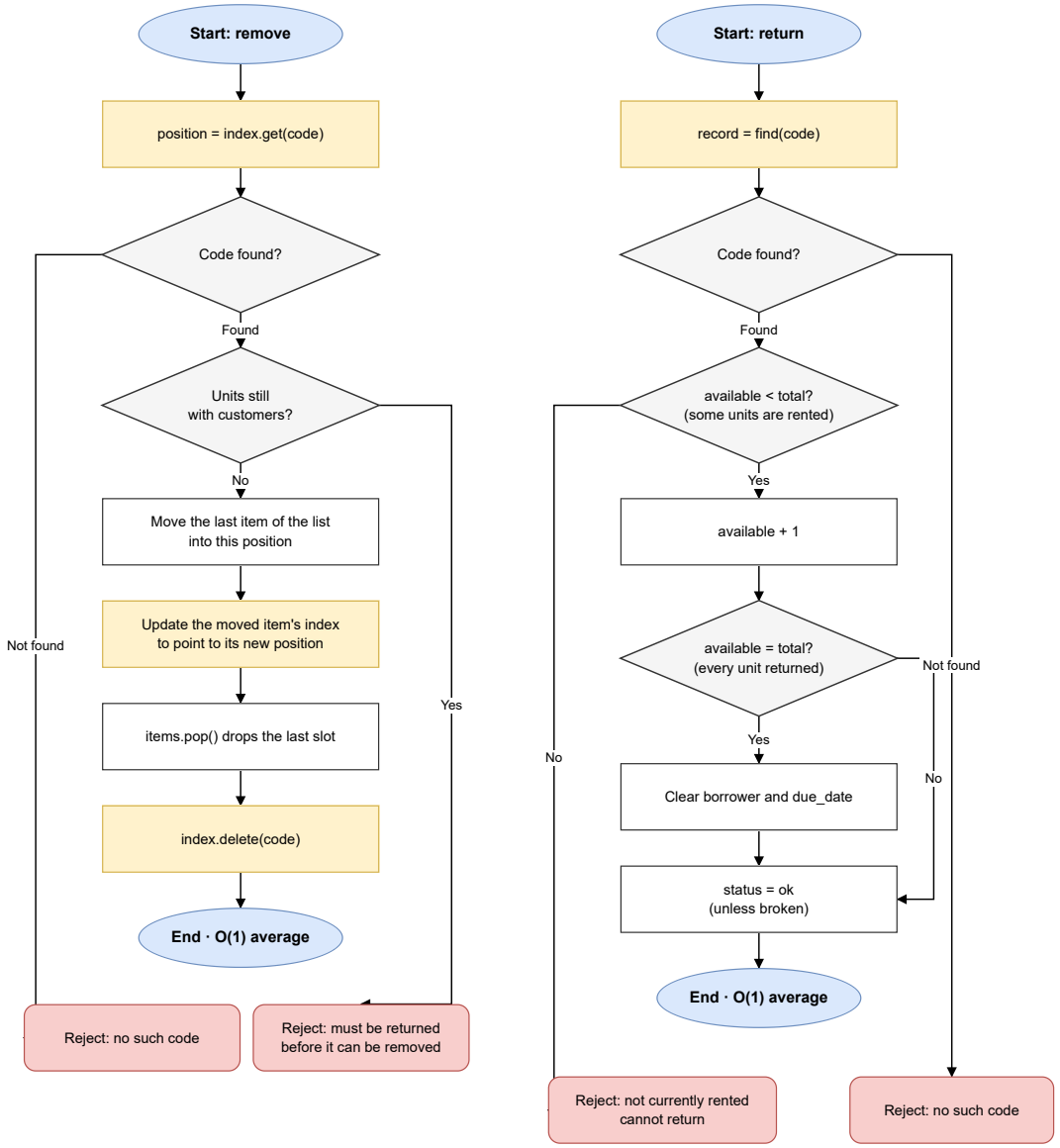
Flowchart 1 — System overview: inventory (list + hash index)



Flowchart 2 — Adding an item (add_item + HashTable.insert)



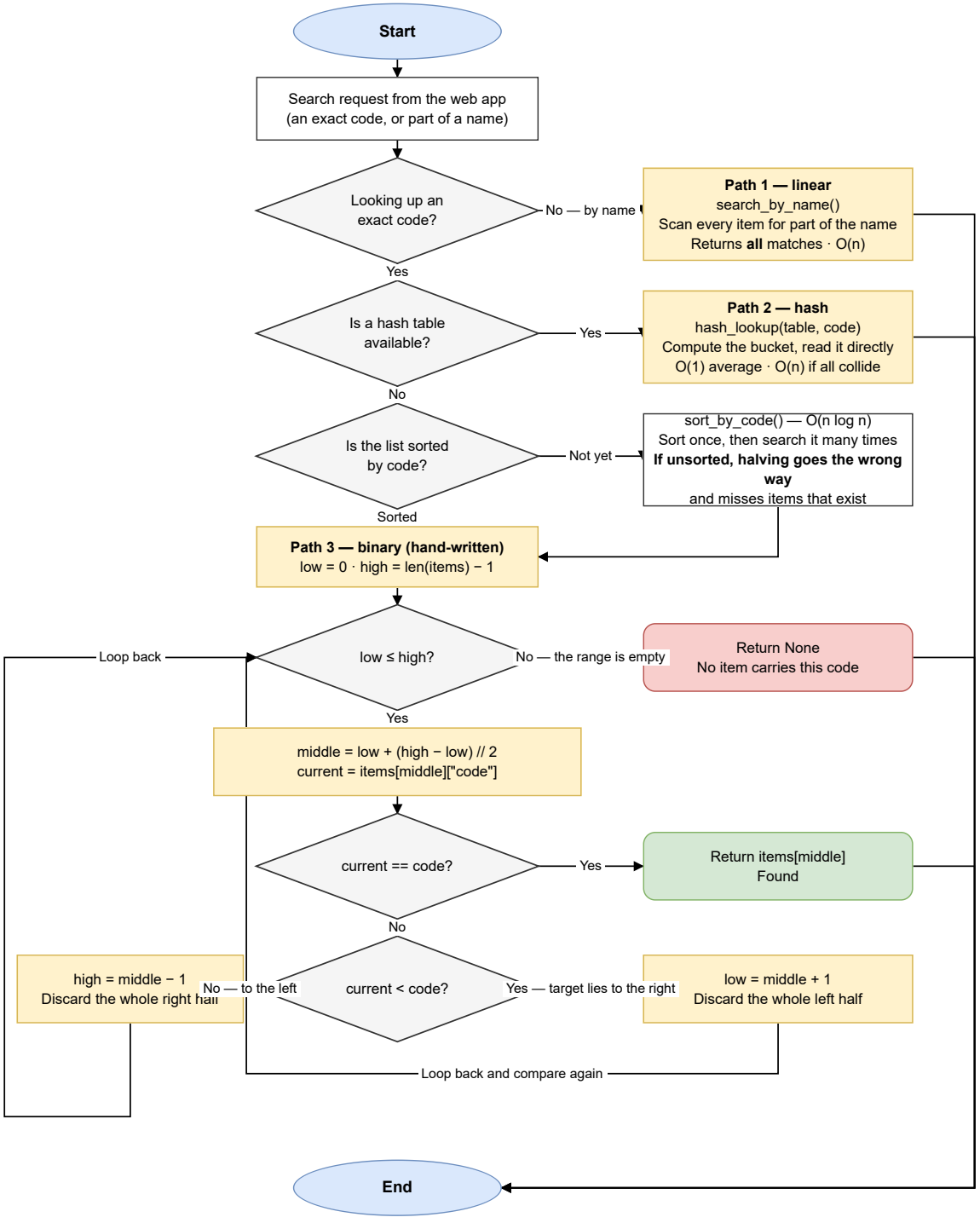
Flowchart 3 — Removing an item (remove_item) and returning (return_item)



Why not items.pop(position) directly?
Deleting mid-list shifts every later item = O(n)
and their positions in the index all become wrong
swap-and-pop updates only one index entry = O(1)

Flowchart 4 — Search: three paths (hash · binary · linear)

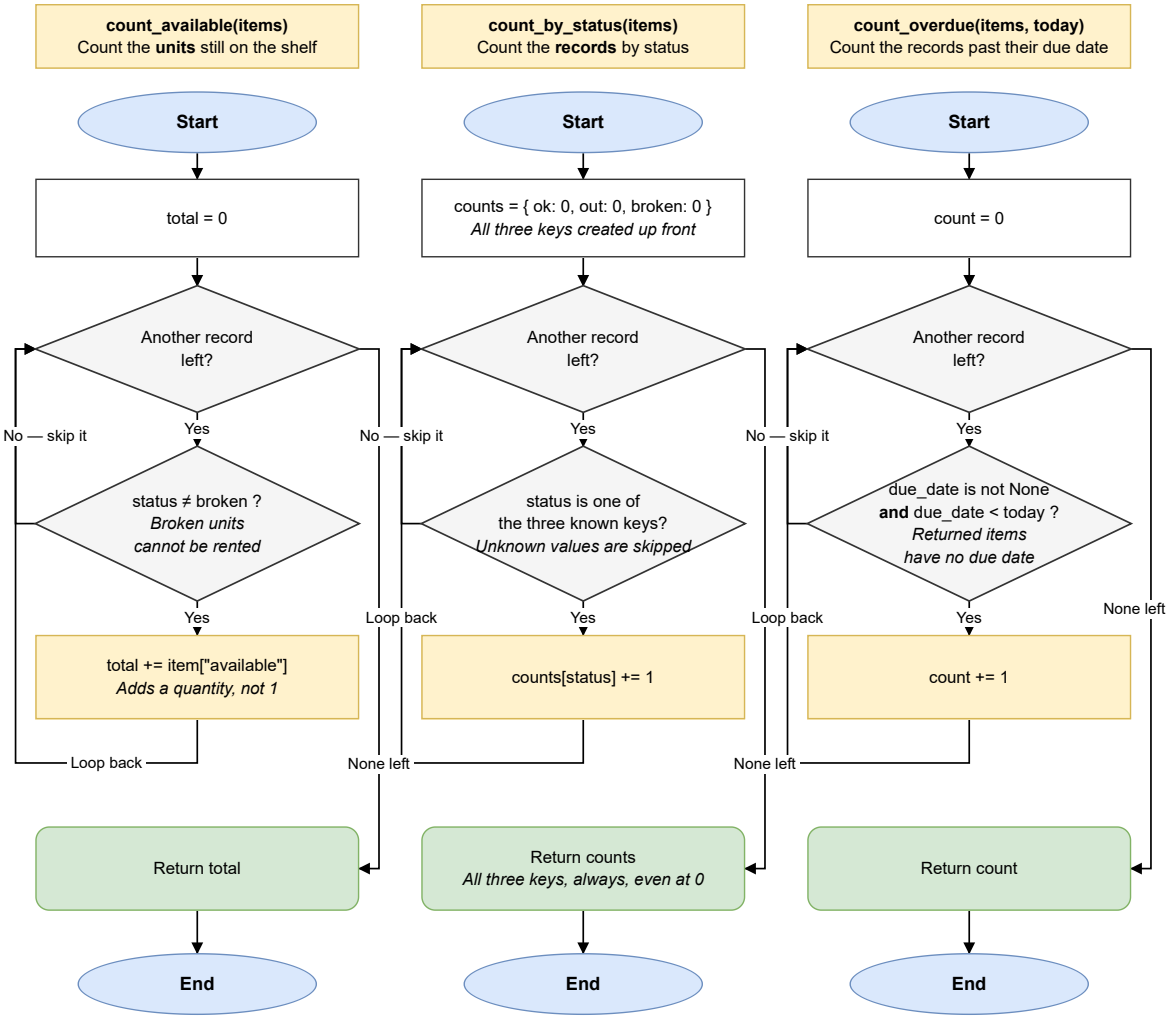
search.py · The path is chosen by the kind of question asked; binary_search, the hand-written one, is drawn in full



Why one round halves the work Each comparison halves what is left to examine, so 1,000 items need about 10 comparisons ($2^{10} = 1{,}024$), against roughly 500 on average for a linear scan · Written as a loop rather than recursion, so it adds no stack frames: memory is O(1), not O(log n)

Flowchart 5 — Counting: count_available · count_by_status · count_overdue

search.py · All three are a single pass over the data — the same skeleton, differing only in the condition and in what is added

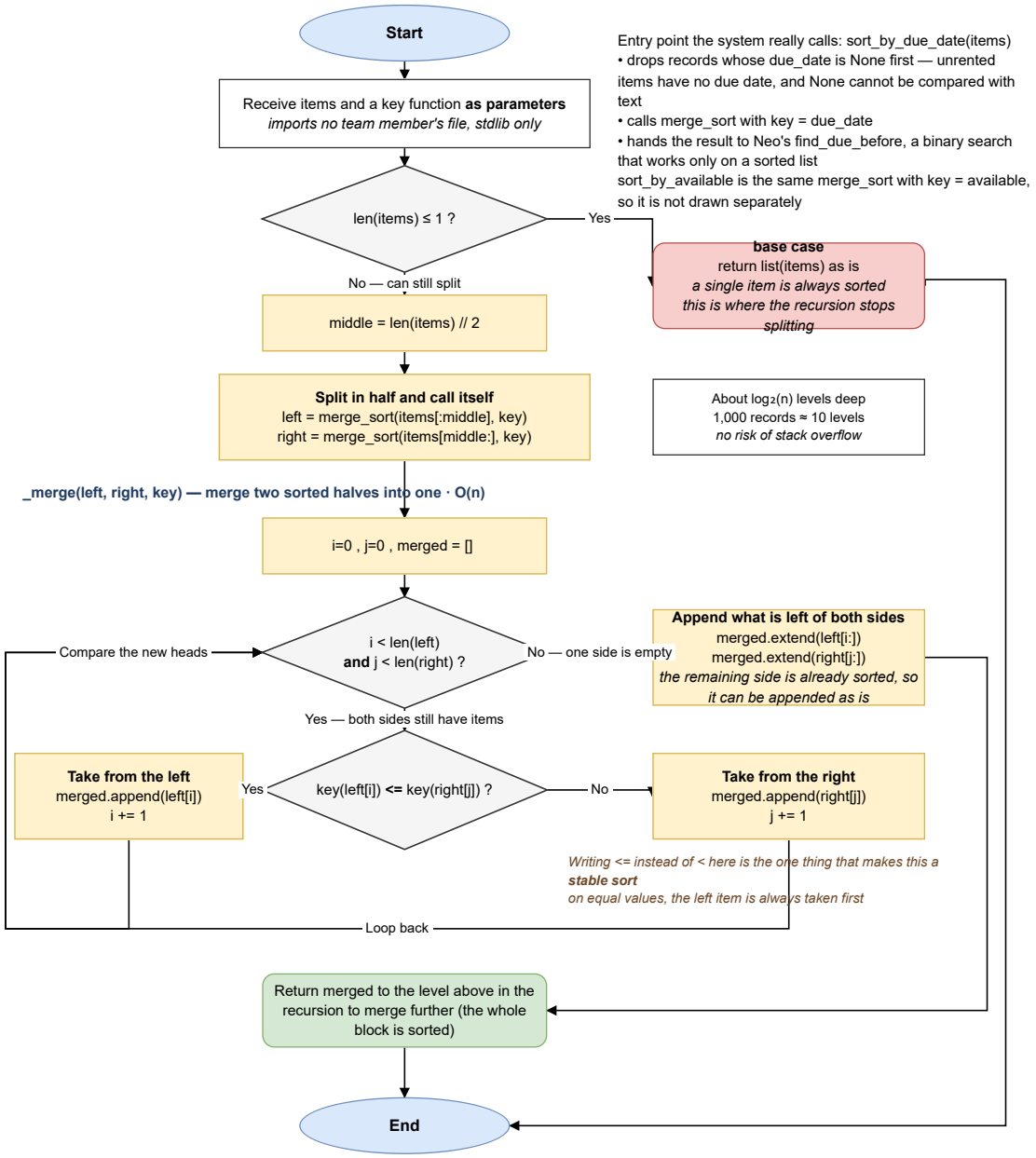


Why all three are O(n), with no shortcut

- Counting differs from searching: there is **no "right one" to stop at**. Every item must be examined before the number is known, so none can be skipped.
- Binary search does not apply — halving needs one target that can be compared as greater or smaller, and counting has no such target.
- A hash table does not help either: it answers "where is this one", never "how many satisfy the condition".
- **Memory is O(1) for all three** — count_by_status holds three fixed slots, which do not grow with n.
- count_by_status produces all three numbers in a **single pass** rather than three separate loops, reading the data three times less.
- Dates compare correctly as plain strings because YYYY-MM-DD runs from the largest unit to the smallest; stored as DD/MM/YYYY this stops being true at once.

Flowchart 6 — Sorting: merge_sort (split in half → merge)

sort_group.py · built as recursion that splits in half until one item is left, unlike hash_table.py, which computes the position straight from the key, and unlike count_* in search.py, which walks the data once



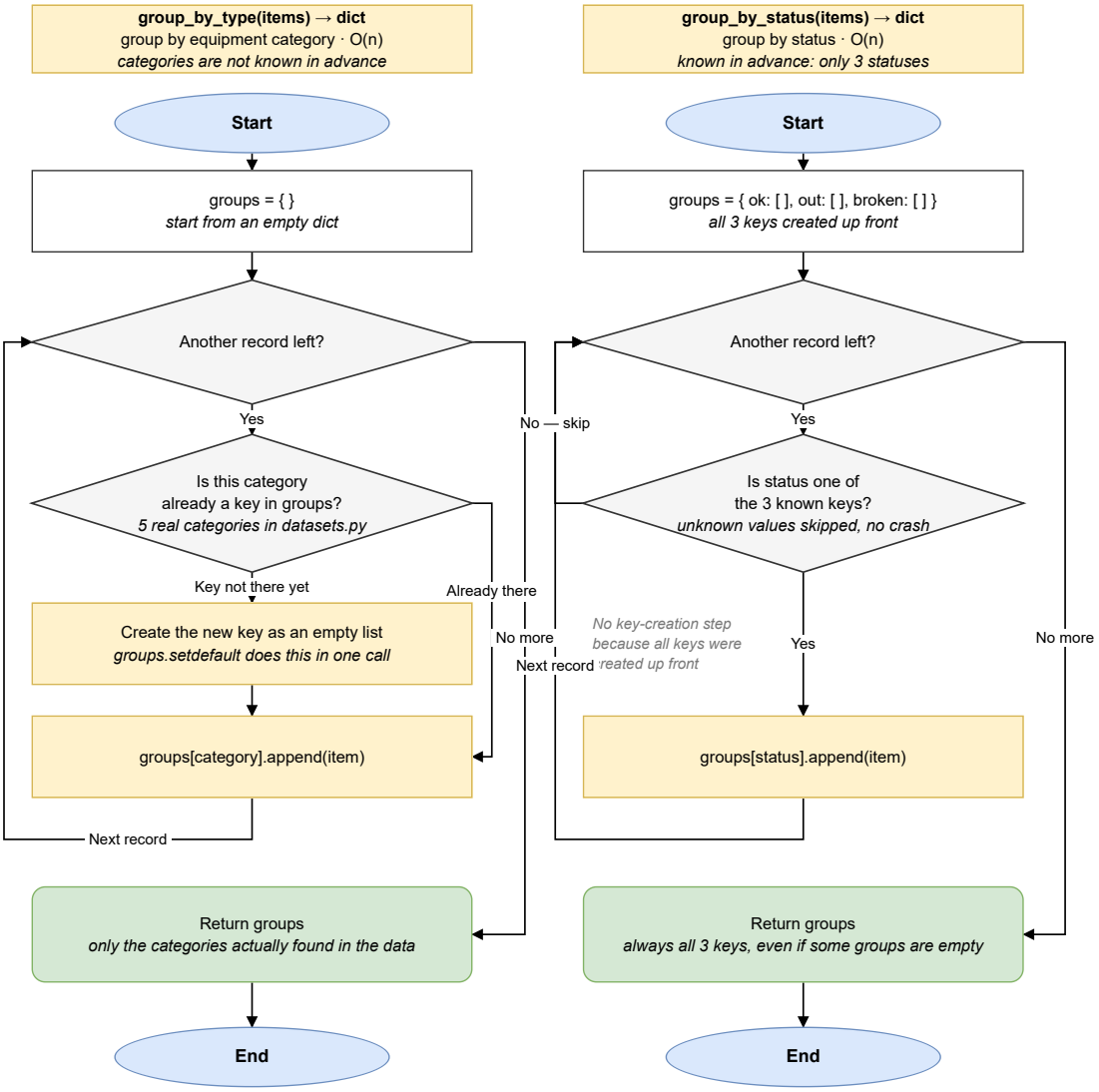
Entry point the system really calls: sort_by_due_date(items)
• drops records whose due_date is None first — unrented items have no due date, and None cannot be compared with text
• calls merge_sort with key = due_date
• hands the result to Neo's find_due_before, a binary search that works only on a sorted list
sort_by_available is the same merge_sort with key = available, so it is not drawn separately

Why splitting in half and merging is faster than swapping pair by pair

• Splitting in half until one item is left gives **log₂(n) levels**, and merging each level costs **O(n)**, for **O(n log n)** in total
• Bubble sort compares every pair over and over, so it is **O(n²)** — measured on set D with 1,000 records, **merge sort is 34.6 times faster than bubble sort**
• Writing **<= instead of <** when comparing the left side makes it a **stable sort**, so records with the same due_date keep their original order instead of being swapped for no reason
• It **always returns a new list and never changes the original**, because someone else may hold the same list, just like sort_by_code in search.py
• The sorted list from this file is what **find_due_before (Neo's binary search)** uses directly; if it were sorted wrongly, the halving would cut the wrong side **with no error at all**

Flowchart 7 — Grouping: group_by_type and group_by_status

sort_group.py · receives items as a parameter, imports no team member's file · walks the data once like count_by_status, but stores whole records in buckets instead of only counting



Why one pass into buckets is faster than searching category by category

- The bucket method reads the data **once**, however many categories there are, so it is **O(n)**
- Searching category by category (first scan for cameras, then scan again for lenses) reads the whole data again **once per category**, so it is **O(n × number of categories)**
- The team's data set has **5 categories** (camera · lens · tripod · light · microphone), so the category-by-category method must read the data **5 times** to get the same result
- **Memory is O(n)** for both functions, because every record is stored in some bucket and none is discarded, unlike counting, which uses **O(1)**
- Unlike **Neo's count_by_status**, which keeps only **counts**, this keeps **whole records**, for when you need to know "what is there", not just "how many"
- Both functions **always return the keys they promise** — group_by_status returns all 3 keys even if some groups are empty, so callers do not have to write .get() guards everywhere, which someone would certainly forget somewhere